AI ENGINEERING
AI-Powered products

MÓDULO 3: Data-driven AI

Sesión 6: Fundamentos de data driven AI - Análisis, formateo y normalización de datos existentes

Mentor: Julián Fernández
LIDR.co, training the Tech Leaders' of the future

Quién os acompaña hoy

Sesión 6 · Módulo 3 · Data-driven AI

Julián Fernández

Informático · fundador de Búho AI

- Transformo operativas complejas en procesos eficientes con IA y software a medida.
- Profesor de IA en LIDR, EBF y UCAM HiTech.
- Enfoque **transparente y ético**: si no sabemos un dato, no lo inventamos — que es justo la metanorma de hoy.

linkedin.com/in/julianfernandez-iaenmurcia

Teaching Assistant

George Kutsia Andguladze

george@lidr.co · dudas técnicas en el canal asíncrono

Cómo es la clase

- Flipped: los 5 articles y el ejercicio ya están hechos.
- **No se teclea código**: se lee el que ya existe y se ejecuta.
- Todo en Python en el servicio IA.

"Hoy no vectorizamos nada: hoy preparamos el corpus para que **merezca** ser vectorizado."

Lo que vamos a ver hoy

En el orden en que viaja un dato: decidir → elegir fuentes → parsear → limpiar → proteger

Apertura · Vuestros números del ejercicio: ¿dónde se rompió el CAG?

Resolución de dudas · Errores del ejercicio y del stack nuevo

1 · Decisión arquitectónica · CAG, RAG o híbrido: una decisión defendible

2 · Catálogo de fuentes · Qué entra al sistema, qué no y por qué

3 · Ingesta · Un Document canónico para todos los formatos

4 · Limpieza y validación · pandas + Pandera: reparar, cuarentena o descartar

5 · PII y GDPR · Pseudonimización con Presidio y mapping table

Cierre · Un corpus listo para vectorizar · puente a la Sesión 7

Al salir de hoy sabréis…

- Defender por qué este proyecto necesita RAG.
- Auditar y catalogar las fuentes de datos.
- Limpiar y validar antes de vectorizar.
- Proteger la PII cumpliendo GDPR.

Cómo trabajamos

Cada pieza: **el fundamento → dónde vive en el código → demo → las decisiones que tenéis que saber defender**.

Todo el código ya está en la rama session_06_live.

¿Qué restricción del CAG se rompió primero en tu ejercicio?

Quiero un número, no una sensación: tokens, ms, $/turn o drift event

1 · Context window
¿Cabía el corpus?

2 · Coste
$/turno con prefix caching

3 · Latencia
P95 frente al SLO

4 · Lost-in-the-middle
recall del dato del centro

Lo que traéis de casa

Alumno	Restricción rota primero	Número concreto	Escenario
1			
2			
3			
4			

¿Alguien detectó lost-in-the-middle?

Es el fallo más sutil: el modelo **siempre responde algo plausible**. Solo aparece si mides el recall del dato que pusiste en el centro.

Mitigarlo = curar el corpus o un retrieval que ponga lo relevante al final → Sesión 7.

"Estos números no son anécdotas: son la justificación arquitectónica del módulo entero."

L1DR

Data-driven AI

La metodología que pone tus datos en el centro de cada decisión de IA

Tus datos
Contexto real de tu negocio

+

Privacidad
Tus datos nunca salen de tu control

+

IA
RAG + orquestación de agentes

=

Ventaja competitiva

Sistemas de IA que razonan exclusivamente sobre tu información privada

No es conectar un chatbot a ChatGPT · es construir tu propio ecosistema de IA

L1DR

Cada problema de la IA genérica, una pieza que lo resuelve hoy

Problema de la IA genérica	Pieza que construimos hoy
Respuestas con datos públicos, no con tu contexto	Catálogo de fuentes + ingesta de tu corpus real
Riesgo de filtración de información confidencial	Pseudonimización con Presidio + mapping table
Alucinaciones sin posibilidad de verificación	Document canónico con metadata trazable a la fuente
Dependencia de proveedores externos	Todo el código en tu servicio IA · soberanía
Imposibilidad de auditar las respuestas	Catálogo versionado en git + política reparar/cuarentena

Lo que vendes en tu web · lo que construimos en las próximas dos horas

L1DR

RAG vs Fine-tuning · cinco dimensiones

Por qué fine-tuning no es alternativa, es capa ortogonal

Dimensión	RAG	Fine-tuning
Actualización	Instantánea	Reentrenamiento
Coste	Bajo y predecible	Alto y variable
Trazabilidad	Fuente citada	No disponible
Alucinaciones	Minimizadas	Frecuentes
Privacidad	Control total	Datos en modelo

RAG es base de conocimiento dinámica · fine-tuning modifica el modelo

Cuando añades un documento nuevo, RAG lo integra al instante

Resolución de errores · 15 minutos

Tres cubetas, en este orden: encoding/formato → dependencias rotas → dudas conceptuales

Formato y encoding

- **UTF-8 vs Latin-1** → `chardet.detect(raw_bytes)` y releer.
- **CSV irregular** → `pd.read_csv(..., sep=";", engine="python", on_bad_lines="warn")`
- **JSON con campos ausentes** → `payload.get("x")` y `x: str | None = None`
- **Fechas en varios formatos** → `pd.to_datetime(s, errors="coerce", dayfirst=True)`

  Es el patrón que veremos en 2.4 (`cleaning/budget_records.py`).

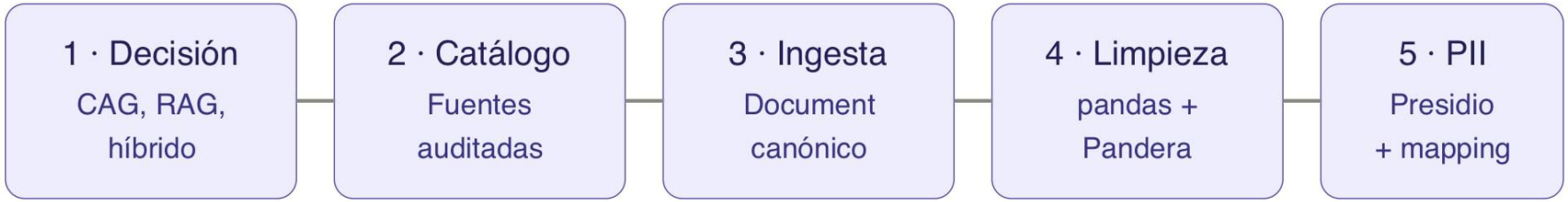
Dependencias del stack

- **es_core_news_md** → sin él Presidio **falla en silencio**. Verificar: `nlp("Laura Fernández firmó en Madrid.").ents` → PER + LOC.
- **unstructured[all-docs]** → ~500 MB la primera vez; para el primer humo, `strategy="fast"`.
- **Pandera v2** → `import pandera.pandas as pa` (no `import pandera as pa`).
- **Pydantic v2** → `@field_validator` + `@classmethod`; `@model_validator(mode="after")`.

El preflight de la sesión comprueba los 8 puntos críticos: `python scripts/preflight_s06.py`

L1DR

El viaje del dato, cinco piezas



Todo en Python en el servicio IA · Rails solo como cliente HTTP

Hoy preparamos el corpus para que merezca ser vectorizado

El primer embedding es Sesión 7

L1DR

Las cuatro restricciones del CAG

Las cuatro tienen que estar en verde a la vez · AND, no OR

1 · Context window

¿Cabe el corpus completo en la ventana del modelo?

2 · Coste

¿El coste/turno con prefix caching es asumible?

3 · Latencia

¿Se queda bajo SLO con el corpus entero en contexto?

4 · Lost-in-the-middle

¿El modelo realmente usa el centro del contexto?

Dónde vive: app/ingestion/architecture.py

La decisión del Article 1, codificada y testeada como un módulo más

Las cuatro restricciones

```
class CAGViability:                # las 4 restricciones
    @property
    def viable(self) -> bool:
        return all([self.context_window_ok,
                    self.cost_ok,
                    self.latency_ok,
                    self.lost_in_the_middle_ok])
```

Proyecto 2 · los números que defendemos

Volumen	**≈ 250 000 tokens**
Refresco	7 días (cierre comercial)
Trazabilidad	obligatoria (contractual)
Acceso	por usuario
Ventana gpt-4o-mini	**128 000 tokens**

Los cuatro ejes

```
def recommend_architecture(corpus, model):
    fits_in_window = (corpus.estimated_tokens
                      <= model.context_window * 0.7)
    cache_friendly = corpus.refresh_frequency_days >= 7
    traceability_doable = not corpus.traceability_required
    access_doable = not corpus.access_control_required
```

Demo · salida real

```
$ python -m app.ingestion.architecture
Corpus 'Proyecto 2': 250,000 tokens, refresh cada 7 días,
  trazabilidad=True, control_acceso=True
Modelo 'gpt-4o-mini': ventana 128,000 tokens

Viabilidad CAG: NO viable
  Restricciones en rojo: context_window, latency,
                         lost_in_the_middle
Recomendación arquitectónica: RAG
```

"250 000 contra 128 000: no cabe ni usando el 70 % útil de la ventana. Game over para el CAG puro."

L1DR

El árbol sobre cuatro ejes

Cada eje en rojo cuenta · uno solo basta para forzar RAG

Volumen

Tokens del corpus vs ventana
útil del modelo (70% segura)

Frecuencia

Días entre refrescos
¿Compatible con prefix caching?

Trazabilidad

¿Hay que citar la fuente
de cada afirmación?

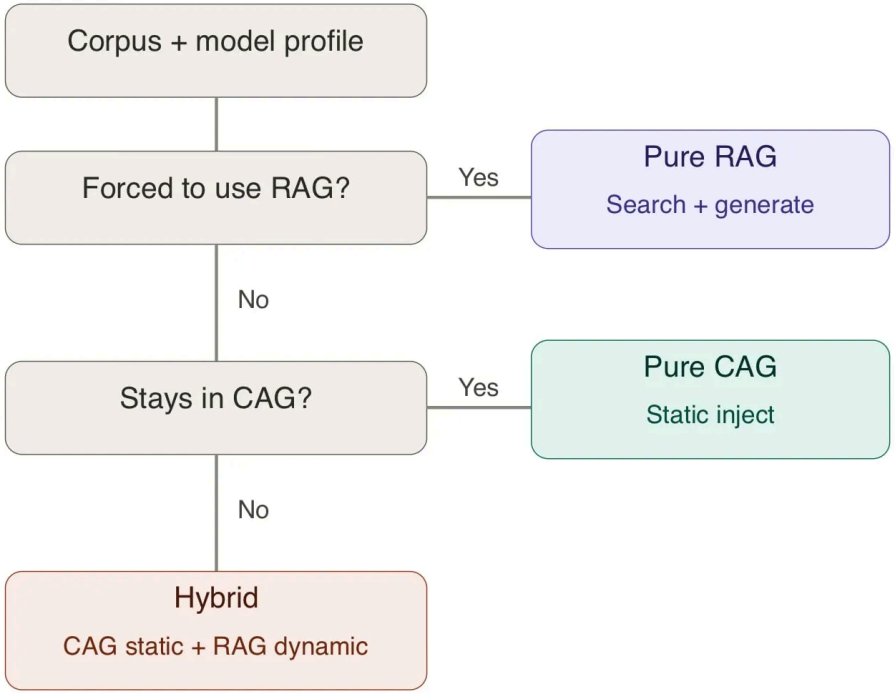
Control de acceso

¿Cada usuario ve solo
sus propios datos?

Fine-tuning no está aquí · es capa ortogonal, no alternativa

La decisión: CAG, RAG o híbrido

Lo que tenéis que poder defender ante un Director de Producto



Los cuatro ejes son AND

Un solo eje en rojo tumba el CAG. Por eso `viable` es un `all([...])`.

Fine-tuning no está en el árbol

Es una capa **ortogonal**: enseña *cómo responder*, no aporta datos nuevos ni resuelve trazabilidad. **Nunca** sustituye a un retrieval mal diseñado.

Tres opciones, no dos

Lo estable (glosario, plantillas, tarifas) por CAG; lo volátil por RAG: la **arquitectura híbrida residual**. Hoy vamos a RAG.

Siguiente → "Decidido RAG, lo primero NO es cargar un parser: es saber **qué** vamos a indexar. Eso es el catálogo."

L1DR

Antipatrón: vectorizar primero, mirar después

Tres modos de fallo, los tres plantados en nuestro corpus de demo

1 · Mezcla de versiones

BUDGET-2024-0005 v1 (30.000 €) y v2 (32.000 €) coexisten · el retriever devuelve ambos

2 · Fuentes podridas

rate_card_2024.xlsx, last_modified 12/01/2024 · indexarlo = tarifas de hace año y medio

3 · Gaps invisibles

transcripcion_2023-09-08_legacy.txt sin speaker tags · el parser produce basura sin avisar

El catálogo es código, no documentación

Tres ficheros pequeños en app/ingestion/catalog/ y un YAML versionado en git

inspect.py · facts, cero opiniones

```
@dataclass
class FolderFacts:
    folder: Path
    file_count: int = 0
    total_size_mb: float = 0.0
    latest_modified: datetime | None
    formats_detected: set[str]
```

models.py · contrato Pydantic v2

```
class CatalogDecision(str, Enum):
    INCLUDE = "include"
    REVIEW  = "review"   # la 3ª
    EXCLUDE = "exclude"

@model_validator(mode="after")
def _decision_requires_reason(...):
    # no include => motivo obligatorio
```

loader.py · deliberadamente trivial

```
def load_catalog(path) -> DataCatalog:
    raw = yaml.safe_load(
        Path(path).read_text("utf-8"))
    return
DataCatalog.model_validate(raw)

# 3 líneas: todo el peso está
# en el modelo, que se testea
```

data/catalog/catalog.yaml

Fuente	Decisión	Motivo
presupuestos_json	**include**	PII confidencial
transcripciones_txt	**review**	la de 2023 sin speaker tags rompe el parser
rate_card_xlsx	**exclude**	actuality=1, fichero del 12/01/2024

Demo · salida real

```
$ python -m app.ingestion.catalog.inspect data/seed
budgets      6 files  formats=json
transcripts  3 files  formats=txt
seed         1 files  latest=2024-01-12  formats=xlsx

$ python -m app.ingestion.catalog.loader \
      data/catalog/catalog.yaml
Sources total: 3 · Sources included: 1
  - presupuestos_json (json)
```

"Tres fuentes, una sola incluida. Esto es lo que enseñáis a un auditor." · excluir es disciplina, no desidia · review evita que lo dudoso se cuele en include

L1DR

Tres capas, responsabilidades separadas

Loader

Produce bytes · no mira dentro · hoy filesystem, mañana HTTP, S3, BD

Parser

Bytes → Documents · lo único que entiende el formato · uno por familia

Normalizer

Reglas de negocio sobre Documents canónicos · es cleaning/ + pii/ del 2.4 y 2.5

El Document canónico es el contrato downstream

Modelo plano · extra para lo específico de cada formato

El Document canónico y los parsers

Un contrato plano para todo lo que sale de la ingesta, venga de JSON, TXT o PDF

documents/models.py

```
class Document(BaseModel):
    model_config =
ConfigDict(extra="forbid")
    id: str
    text: str
    metadata: DocumentMetadata
    # lo específico de formato
    # va en metadata.extra
```

parsers/protocol.py · Protocol, no ABC

```
@runtime_checkable
class Parser(Protocol):
    supported_formats:
ClassVar[set[str]]
    def parse(self, blob, context
              ) -> Iterable[Document]:
...
```

parsers/registry.py

```
registry.register(BudgetJsonParser())
registry.register(TranscriptTxtParser())
# XLSX/DOCX/PDF: repo de referencia
# añadir un formato = una línea
```

budget_json.py · markdown, no json.dumps

`text = _render_budget_markdown(payload)`

El embedder embebe **prosa**: "¿quién es el cliente?" se parece más a `## Cliente - Nombre: Acme` que a `{"client_name":"Acme"}`.
Una decisión de retrieval que adelantamos al parser.

transcript_txt.py · un formato, dos modos

- **Tagged** (≥ 3 líneas `[hh:mm:ss] Speaker:`) → un Document por turno con `speaker` y `timestamp`.
- **Legacy** → un Document por bloque, **sin speaker: no lo fabrico**.

"Mejor admitir que no sé el speaker que inventármelo."

extra="forbid": si un parser cuela un campo no declarado, Pydantic falla · **error fuerte > error silencioso** · el loader solo da bytes, el parser no sabe de dónde vienen

L1DR

Offline vs online · la separación del Article 1

Pipeline offline

Indexación · hoy lo construimos

POST /ingestion/runs → 202

Background task · minutos OK

GET /ingestion/jobs/{id}

El cliente hace polling

Pipeline online

Query · Sesión 7

POST /query → 200

Síncrono · bajo SLO < 3 s

El usuario espera

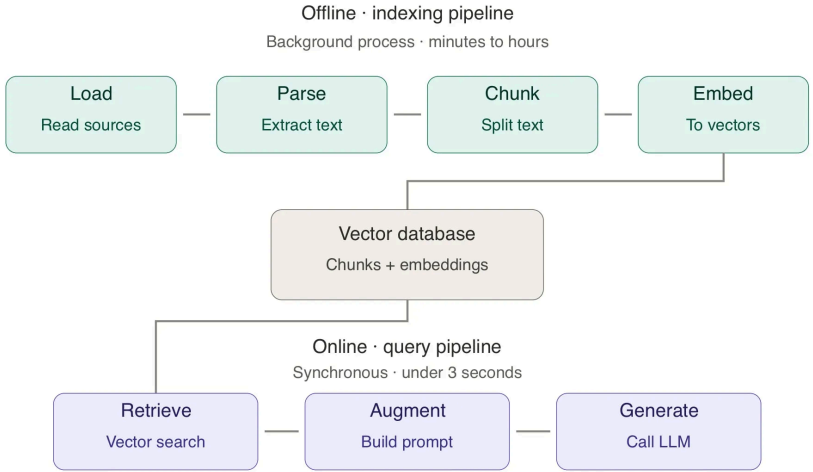
No bloquear nunca

Indexar un corpus tarda minutos · llamada síncrona = cliente tirado

Por eso indexación es 202 + polling, query será 200 directo

La ingesta es offline: 202 + polling

Indexar tarda minutos; una llamada síncrona tira al cliente. El query de la Sesión 7 sí será síncrono



Demo · salida real

```
$ curl -X POST localhost:8000/api/v1/ingestion/runs \
       -d '{"source_name": "presupuestos_json"}'
{"job_id": "e417f7f9-...", "status": "pending"}       ← 202

$ curl localhost:8000/api/v1/ingestion/jobs/$JID
{"status": "completed", "documents_count": 6,
 "error_message": null}

# ramas de error
transcripciones_txt (review)  → HTTP 400
rate_card_xlsx (exclude)      → HTTP 400 source_not_included
no_existe                     → HTTP 404 unknown_source
```

Router fino

Valida contra el catálogo, crea el job, dispara el `BackgroundTask` (con su propia sesión de BD) y devuelve **202**.

Cliente Rails ilustrativo · ingestion_client.rb

Solo ilustra el contrato: **cualquier cliente HTTP funciona** (curl, httpx, fetch). El servicio Python es el autoritativo.

XLSX, DOCX y PDF siguen exactamente este patrón: los tenéis en el repo de referencia.

L1DR

Cuatro familias de suciedad · plantadas en el seed

1 · Heterogeneidad

currency: "EUR" vs "eur"

en 0001 vs 0002

→ se repara en limpieza

2 · Disguised nulls

client_name: "TBD"

en 0003

→ va a cuarentena

3 · Duplicados divergentes

BUDGET-2024-0005 v1: 30.000

BUDGET-2024-0005 v2: 32.000

→ keep last by signed_at

4 · Out-of-range

total_amount: -50000

en 0004

→ descarte (Check.ge(0))

6 ficheros → limpieza → 5 filas → validación → 3 valid + 1 cuarentena + 1 descarte

Limpiar y validar son dos pasos distintos

Limpieza permisiva (repara lo reparable) · validación estricta (decide qué entra)

1 · Limpieza

```
# cleaning/budget_records.py  — repara, no juzga
df["signed_at"] = pd.to_datetime(df["signed_at"],
                      errors="coerce", dayfirst=True)
df = (df.sort_values(["budget_id", "signed_at"])
        .drop_duplicates(subset=["budget_id"], keep="last"))
```

2 · Contrato de contenido

```
# cleaning/schemas.py  — Pandera v2
import pandera.pandas as pa
"total_amount": Column(float,
     checks=[Check.ge(0), Check.le(10_000_000)]),
"client_name":  Column(str, nullable=True),
strict=True, coerce=False
```

3 · Política

```
# cleaning/policy.py  — enruta a tres cubetas
QUARANTINE_CHECKS = {"not_nullable", "no_default",
                     "column_in_dataframe"}
DISCARD_CHECKS    = {"in_range", "greater_than_or_equal_to",
                     "isin", "str_matches"}
validated = BudgetRecord.validate(df, lazy=True)
```

Reparar · cuarentena · descartar

Reparar	sin pérdida semántica, sin humano
Cuarentena	grave pero útil tras revisión: no entra al RAG, se guarda con motivo
Descartar	contaminación clara: fuera, con log

Para defender

`errors="coerce"`: no excepciones, la validación decide · dedup por hash distingue duplicado exacto de divergencia · "keep last by signed_at" es **política de negocio** documentada en código · `lazy=True` recoge **todos** los errores para poder enrutar · `strict=True` siempre: lo que se relaja es la política, no el contrato.

Lo que sale de verdad en la demo

Ejecución real del código de la sesión

Demo · salida real

```
$ python scripts/demo_cleaning_s06.py
Ficheros leídos: 6
UserWarning: Parsing dates in %Y-%m-%d format
  when dayfirst=True was specified.
Filas tras limpieza + dedup: 5

Report:
  input_rows: 5
  valid_rows: 3
  quarantined_rows: 1
  discarded_rows: 1

Descartadas:
  BUDGET-2024-0004   -50000

En cuarentena:
  BUDGET-2024-0002   BetaNorte Distribución
```

6 ficheros → 5 filas

El dedup colapsó las dos versiones de **BUDGET-2024-0005** y se quedó con la v2 (firmada después). Política en código, no en un acta.

¿Por qué va a cuarentena el 0002 y no el "TBD"?

Su fecha es `12/03/2024`; las demás, `2024-05-02`. pandas deduce el formato de la **primera** fila y la del 0002 no encaja → `NaT` → falla `not_nullable`. El 0003 ("TBD") pasa porque `client_name` admite nulos.

La lección

`dayfirst=True` no bastó: **la limpieza no pudo reparar esa fecha y la validación la cazó**. Por eso son dos pasos distintos.

Siguiente → "Datos limpios y validados. **Pero todavía con PII en claro.** Lo arreglamos ahora, y es lo más delicado del módulo."

L1DR

Tres modos de filtración semántica vía RAG

El control de acceso del backend no protege contra ninguno · defender antes del embedding

1 · Directa

El nombre/email/id aparece literal en la respuesta · pseudonimizamos antes de embedder

2 · Por agregación

CP + edad + género reidentifican aunque cada campo sea inocuo · pseudonimizar cuasi-ids

3 · Por inferencia · la más peligrosa

El modelo adivina desde el lenguaje · pseudonimizar no la frena · queda abierta como teoría

Empezar por el fallo: Presidio en inglés

Por qué hay que configurar el español de forma explícita

Lo que pasa por defecto

Presidio monta spaCy **en inglés**. Sobre "Laura Fernández" o "Javier Romero" devuelve **cero** entidades PERSON.

Y **falla en silencio**: el pipeline parece funcionar y en producción alguien pregunta "¿qué dijo Laura Fernández?" y el LLM responde con el nombre sin pseudonimizar.

pii/analyzer.py · la corrección

```
@lru_cache                          # spaCy tarda ~300 ms
def build_analyzer() -> AnalyzerEngine:
    provider = NlpEngineProvider(nlp_configuration={
        "nlp_engine_name": "spacy",
        "models": [{"lang_code": "es",
                    "model_name": "es_core_news_md"}]})
    engine = AnalyzerEngine(
        nlp_engine=provider.create_engine(),
        supported_languages=["es"])
    engine.registry.add_recognizer(BudgetIdRecognizer())
    engine.registry.add_recognizer(ClientCodeRecognizer())
    return engine
```

Recognizers del dominio

`BUDGET-\d{4}-\d{4}` y `CLI-\d{4}`, score 0.95 (regex muy específico).
Presidio trae PERSON, EMAIL o LOC de fábrica; nuestros ids, no. Sin recognizer propio se vectorizan en claro.

Con la configuración en español

`[PERSON Laura Fernández, LOC Madrid]` ✔

El preflight lo comprueba con esa misma frase: `detected PER entities: ['Laura Fernández'].`

L1DR

Mismo valor → mismo pseudónimo, siempre

pii/pseudonymizer.py + mapping table en Postgres (pgvector instalado pero dormido)

Tres líneas aha

```
self._generators = {
    "PERSON": self._faker.name,
    "EMAIL_ADDRESS": self._faker.ascii_company_email, ...}
results.sort(key=lambda r: r.end, reverse=True)  # der→izq
original_hash = self._hash(original_value)  # HMAC-SHA256
```

persistence/models.py · la mapping table

```
UniqueConstraint("entity_type", "original_hash",
                 name="uq_mappings_entity_hash")
# se guarda el hash, NUNCA el valor en claro
```

Demo · salida real (transcripción BetaNorte)

```
$ python scripts/demo_pii_s06.py
[09:15:02] Pánfilo del Barros: Buenas. Estamos revisando
el presupuesto BUDGET-2026-3206 firmado en marzo, código
de cliente Apolinar Montero Verdejo. ...
--- Mappings aplicados (20) ---
  PERSON         → Purificación Rivero Arjona
  EMAIL_ADDRESS  → dalila38@instalaciones.org
  ORGANIZATION   → REDACTED-7c65c1e5
  LOCATION       → Teruel
```

Por qué así

- Generador **por tipo**: un nombre se cambia por un nombre, no por `[PERSON]`.
- Reemplazo de derecha a izquierda: los offsets no se desplazan.
- El UNIQUE hace la concurrencia segura sin locks.

Falsos positivos (saldrán)

Ejemplo real: el código de cliente se reemplazó por un nombre de persona. No es un bug, es el trade-off de Presidio en español. En producción: subir el umbral, deny-list o `es_core_news_lg`.

La demo usa un store en memoria: la tabla de Postgres sale vacía.

Derecho al olvido (GDPR art. 17): por qué la mapping table es arquitectura

Sin ella, borrar a un cliente es imposible: está embebido en miles de chunks

1 · Buscar en la mapping store todos los pseudónimos del solicitante (con sus variantes).

2 · Localizar en el índice los chunks con esos pseudónimos o sus documentos.

3 · Eliminar esos chunks del índice vectorial.

4 · Borrar sus entradas de la mapping store: si reaparece, recibe un pseudónimo nuevo.

5 · Registrarlo en un audit log: atendido en plazo y forma.

Pseudonimizar, no anonimizar

`[PERSON]` genérico tira la similitud un **15–25 %**. Un nombre verosímil y consistente preserva la estructura semántica.

Lo que NO frena

La filtración **por inferencia** (estilo, jerga, dialecto → identidad): la señal está en el lenguaje, no en los identificadores. Queda abierta como teoría.

El control de acceso de Rails no protege contra ninguno de los tres modos.

Cinco piezas: el corpus ya es defendible

Ante legal, comercial, técnico y regulatorio

#	Pieza	Dónde vive	Article
1	Decisión arquitectónica (RAG, híbrido residual)	`app/ingestion/architecture.py`	1
2	Catálogo versionado, 3 fuentes auditadas	`data/catalog/catalog.yaml` + `catalog/`	2
3	Ingesta + Document canónico + endpoint asíncrono	`loaders/ parsers/ orchestrator.py`	3
4	Limpieza + Pandera + reparar/cuarentena/descartar	`app/ingestion/cleaning/`	4
5	Pseudonimización GDPR-grade + mapping table	`app/ingestion/pii/` + `persistence/`	5

Eso es data-driven AI

La exclusión del XLSX tiene motivo, la cuarentena tiene motivo, la política de dedup tiene motivo, el modelo de Presidio en español tiene motivo. Todo versionado en git.

Sesión 7 →

Chunking, embeddings y espacio vectorial. Partimos los Documents, los embebemos y los metemos en pgvector, que se activa con **una sola migración Alembic**.

✋Tiempo para el feedback con ROTI

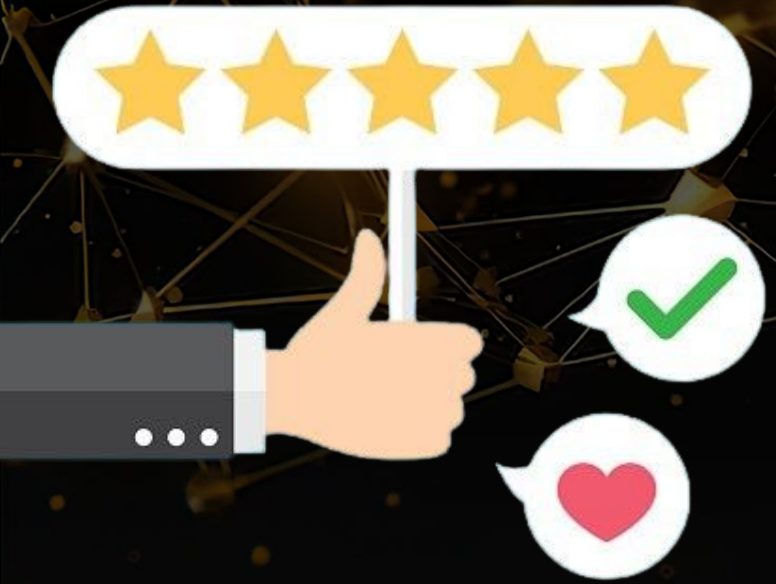
¿Cuánto valor te ha agregado esta sesión?



Absolutamente nada	Poco	Suficiente	Bastante	Más de lo que esperaba

LIDR

ROTI



¡GRACIAS!

LIDR.co, AI-Powered Careers

@lidr_ai

#followtheLIDR